

Unidad Didáctica

Gestión de Proyectos, Unidad IV

Contenido

Índice de Contenido 2

Introducción 3

Desarrollo de Contenidos 4

 Espectro administrativo 4

 Personal 13

 Producto..... 19

 Proceso 28

 Proyecto 38

Bibliografía y Webgrafía 48

Glosario 49

Introducción

La Unidad IV integra y eleva los aprendizajes previos del curso: mientras que las unidades anteriores te proporcionaron marcos metodológicos, prácticas ágiles (SCRUM) y técnicas de verificación (pruebas), esta unidad se centra en la **gestión y gobernanza** del desarrollo de software. Aquí aprenderás a articular la planificación estratégica, la organización del equipo, la definición y evolución del producto, la formalización de procesos y la dirección de proyectos de manera coherente y medible, de modo que el esfuerzo técnico se traduzca en resultados sostenibles y de valor para los usuarios y la institución.

A lo largo de la unidad se prioriza el pensamiento crítico y la toma de decisiones basada en evidencia: definirás objetivos claros, seleccionarás indicadores relevantes (KPIs), evaluarás riesgos y aplicarás mecanismos de control que permitan responder a la incertidumbre sin perder la agilidad. El objetivo práctico es que al finalizar los estudiantes sepan diseñar y gestionar un proyecto de software completo —desde el charter hasta el cierre—, entregando un producto que cumpla requisitos técnicos, normativos y, sobre todo, necesidades reales de los usuarios.

Esta unidad adopta un enfoque aplicado y contextualizado: los conceptos se trabajarán con ejemplos y actividades prácticas (pilotos, backlogs priorizados, plantillas de gestión y métricas reales) que faciliten su transferencia a escenarios locales como los CUR de la Universidad Nacional Politécnica o empresas tecnológicas de la región.

Espectro administrativo

El **espectro administrativo** en ingeniería del software representa el **conjunto de prácticas gerenciales** que permiten planificar, organizar, dirigir, coordinar y controlar los recursos humanos, técnicos y financieros involucrados en la creación de un sistema de información o aplicación informática.

El término “espectro” es pertinente porque abarca **una gama completa de actividades** que van desde la visión estratégica (alineación con los objetivos de la organización o cliente) hasta la supervisión operativa diaria (seguimiento de tareas, control de riesgos y métricas de calidad).

Desde la perspectiva científica y normativa, el **espectro administrativo** se apoya en estándares internacionales como:

- **ISO/IEC/IEEE 12207:2017**, que define los procesos del ciclo de vida del software, incluyendo los procesos de gestión y soporte.
- **PMBOK® (Project Management Body of Knowledge, PMI)**, que estructura las áreas de conocimiento en dirección de proyectos (integración, alcance, tiempo, costo, calidad, recursos, comunicaciones, riesgos, adquisiciones y stakeholders).
- **CMMI (Capability Maturity Model Integration)**, que orienta la madurez organizacional para la gestión de proyectos de software.

Cada uno de estos marcos aporta componentes que, combinados, forman un sistema de administración coherente y cuantificable.

Dimensiones del espectro administrativo

El espectro administrativo puede dividirse en **cuatro dimensiones esenciales**:

1. **Planificación estratégica**
2. **Gestión operativa**
3. **Control y monitoreo**
4. **Gestión de riesgos**

Estas no son fases secuenciales, sino **dimensiones interdependientes y recurrentes** dentro del ciclo de vida del proyecto.

1. Planificación estratégica

a) Definición

La **planificación estratégica** consiste en definir **qué se va a hacer, por qué, con qué recursos y en qué tiempo**, estableciendo una alineación directa entre los objetivos del proyecto y los objetivos de la organización.

En ingeniería del software, esto se traduce en:

- Definir el **alcance funcional** (qué módulos o funcionalidades tendrá el sistema).
- Determinar los **objetivos de calidad** (rendimiento, seguridad, mantenibilidad).
- Asignar **recursos humanos y tecnológicos** (desarrolladores, infraestructura, software de apoyo).
- Establecer el **presupuesto y cronograma**.

“La planificación estratégica en proyectos de software es la traducción de la estrategia organizacional en un conjunto coherente de decisiones técnicas, logísticas y humanas” (Pressman & Maxim, *Software Engineering: A Practitioner’s Approach*, 9th Ed., 2020).

b) Herramientas y técnicas

- **Estructura de desglose del trabajo (WBS, Work Breakdown Structure):** permite dividir el proyecto en entregables más pequeños y manejables.
- **PERT/CPM:** para estimar tiempos y rutas críticas.
- **Análisis de interesados (Stakeholder Analysis):** identificar quiénes influyen o son afectados.
- **Matriz de asignación de responsabilidades (RACI):** clarifica roles.

c) Ejemplo práctico (contexto Nicaragua)

Proyecto: *Sistema de control de asistencia docente para la Universidad Politécnica de Nicaragua.*

En la planificación estratégica se definirían:

- **Objetivo:** crear una plataforma que permita registrar, validar y analizar la asistencia docente de todos los CUR en tiempo real.
- **KPIs estratégicos:**
 - Reducción del 70% del tiempo invertido en consolidar reportes.
 - Error en datos de asistencia < 1%.
- **Presupuesto estimado:** C\$180,000.00.
- **Cronograma global:** 6 meses con entregas incrementales cada 4 semanas.

2. Gestión operativa

a) Definición

La **gestión operativa** transforma la planificación en acción. Se ocupa de coordinar los recursos y tareas cotidianas necesarias para cumplir los objetivos definidos en la estrategia.

En un entorno ágil, esta gestión se realiza a través de:

- **Sprints o iteraciones.**
- **Tableros Kanban o Scrum boards.**
- **Reuniones diarias (daily standups).**
- **Gestión de incidencias y cambios.**

b) Actividades clave

1. **Asignación de tareas** según capacidades del personal.
2. **Seguimiento del progreso** mediante herramientas de control (Jira, Trello, Azure Boards).
3. **Gestión de dependencias y bloqueos** (impediments).
4. **Coordinación entre equipos distribuidos** (por ejemplo, entre los CUR León, Chontales y Managua).

c) Ejemplo aplicado

Si el proyecto involucra a varios CUR, la gestión operativa debe sincronizar tareas a través de herramientas colaborativas.

Por ejemplo:

- Trello puede organizar tableros por CUR ("CUR Managua", "CUR Estelí").
- Jira centraliza incidencias y tareas globales.
- Slack o Teams integra la comunicación diaria.

Así, el espectro administrativo no solo se limita a documentos, sino que se materializa en **flujos digitales de trabajo**.

3. Control y monitoreo

a) Definición

El **control y monitoreo** consiste en medir, comparar y corregir el desempeño del proyecto respecto al plan original.

Se fundamenta en la máxima de Peter Drucker:

“Lo que no se mide, no se puede gestionar.”

b) Elementos del control administrativo

- **Indicadores clave de desempeño (KPIs):**
 - *Velocidad (Velocity)*
 - *Defect Density*
 - *Lead Time*
 - *Cumplimiento del cronograma (% de tareas completadas en plazo)*
 - *Satisfacción del cliente o usuario final*
- **Mecanismos de retroalimentación:**
 - Reuniones de revisión del sprint (*Sprint Review*).
 - Auditorías de calidad internas.
 - Revisiones de código (*code reviews*).
 - Evaluaciones de satisfacción de usuario.

c) Herramientas y reportes

- **Jira** genera reportes automáticos de burndown y velocity.
- **Microsoft Project** permite calcular variaciones de cronograma (SV, Schedule Variance) y de costo (CV, Cost Variance).
- **Power BI o Grafana** visualizan métricas en tiempo real.

d) Fórmulas útiles

- **Schedule Variance (SV)** = $EV - PV$ (Earned Value – Planned Value).
- **Cost Variance (CV)** = $EV - AC$ (Earned Value – Actual Cost).
- **Performance Index (SPI o CPI)** = EV/PV o EV/AC .
 - Si $SPI < 1 \Rightarrow$ retraso.

- Si $CPI < 1 \Rightarrow$ sobrecosto.

e) Ejemplo numérico (breve)

Supón:

- Valor planificado (PV): C\$100,000
- Valor ganado (EV): C\$80,000
- Costo real (AC): C\$90,000

Entonces:

- $SV = EV - PV = -20,000 \rightarrow$ retraso.
- $CV = EV - AC = -10,000 \rightarrow$ sobrecosto.
- $SPI = 0.8 \rightarrow$ 20% de retraso respecto al plan.
- $CPI = 0.89 \rightarrow$ eficiencia de costo del 89%.

Estos indicadores permiten tomar decisiones basadas en evidencia, no en percepciones.

4. Gestión de riesgos

a) Definición

La **gestión de riesgos** identifica, analiza, planifica y supervisa los eventos potenciales que pueden afectar negativamente (o positivamente) el desarrollo del proyecto.

b) Tipos de riesgos comunes en software

<i>Categoría</i>	<i>Ejemplo</i>
<i>Técnico</i>	Fallas en la integración de módulos o APIs externas.
<i>Organizacional</i>	Cambio de prioridades institucionales.
<i>Recurso humano</i>	Rotación de personal o falta de competencias.
<i>Externo</i>	Cortes eléctricos, inestabilidad política, problemas con proveedores.
<i>Regulatorio</i>	Cambios en políticas de protección de datos.

c) Ciclo de gestión de riesgos

1. **Identificación:** tormenta de ideas, revisión de lecciones aprendidas, entrevistas.
2. **Análisis:** asignar probabilidad (P) e impacto (I).
3. **Evaluación:** calcular la exposición al riesgo ($R = P \times I$).
4. **Tratamiento:** elegir estrategias (mitigar, transferir, aceptar, evitar).
5. **Monitoreo:** revisar periódicamente y actualizar el registro.

d) Ejemplo

Riesgo: "Demora en la validación de requisitos por falta de disponibilidad de los docentes."

- Probabilidad: Alta (0.8)
- Impacto: Medio (C\$20,000 equivalentes en tiempo perdido)
- Exposición: $R = 0.8 \times 20,000 = \text{C\$}16,000$
- Mitigación: planificar sesiones virtuales y grabarlas para revisión asincrónica.

e) Herramientas

- Registro de riesgos (hoja Excel o módulo en Jira).
- Matriz de calor (heat map) para priorización visual.
- Análisis Monte Carlo (en proyectos grandes) para simular escenarios.

Integración del espectro administrativo con metodologías ágiles

En contextos modernos, **el espectro administrativo no reemplaza a las metodologías ágiles**: las complementa.

Mientras **Scrum** regula el flujo operativo de trabajo, el espectro administrativo asegura que exista **gobernanza** (KPIs, presupuesto, riesgos, control de alcance).

Por ejemplo:

- La **planificación estratégica** se refleja en el *Product Goal* de Scrum.
- La **gestión operativa** se materializa en los *sprints* y el *Daily Scrum*.
- El **control** se ejerce con el *Sprint Review* y el *Burndown Chart*.
- La **gestión de riesgos** se revisa en la *Retrospective*.

El resultado es una combinación poderosa: **agilidad con disciplina**, el equilibrio ideal entre adaptabilidad y control.

Caso de estudio

Proyecto: Plataforma de Gestión de Investigación Académica (PGIA)

Institución: Universidad Politécnica de Nicaragua

Objetivo: digitalizar el proceso de registro, aprobación y seguimiento de proyectos de investigación de docentes y estudiantes.

Aplicación del espectro administrativo:

<i>Dimensión</i>	<i>Ejemplo concreto</i>
<i>Planificación estratégica</i>	Se definen tres objetivos: (1) reducir tiempo de aprobación de proyectos a la mitad; (2) garantizar trazabilidad documental; (3) integrar módulo de reportes PDF automáticos.
<i>Gestión operativa</i>	Tres equipos (por facultad) trabajan en módulos distintos bajo coordinación central; Trello y GitHub Projects como herramientas.
<i>Control y monitoreo</i>	KPIs: % de avance de entregas, defectos abiertos, tiempo medio de revisión de código, satisfacción del usuario (encuesta).
<i>Gestión de riesgos</i>	Riesgos identificados: fallos en servidores, rotación de estudiantes desarrolladores, retrasos por validación académica. Mitigación: respaldo en la nube, documentación robusta y reuniones semanales con directores de facultad.

Este caso muestra cómo el espectro administrativo es **transferible y aplicable**, incluso en instituciones educativas con recursos limitados.

- El **espectro administrativo** es la arquitectura invisible que sostiene la ingeniería del software profesional.
- Sin él, los proyectos se vuelven esfuerzos caóticos: sprints sin dirección, entregas sin control y equipos sin propósito.
- Con él, cada decisión técnica se alinea a un objetivo estratégico, y cada línea de código se traduce en valor organizacional.

Como afirmaba Frederick Brooks en *The Mythical Man-Month*:

“Ninguna cantidad de programación rápida puede compensar una planificación deficiente.”

El buen ingeniero de software no solo codifica, **administra conocimiento, riesgo y estrategia**. Ese es el verdadero sentido del espectro administrativo.

Personal

"Un proyecto de software no fracasa por falta de tecnología, sino por una gestión inefectiva de las personas que la hacen posible."

— Barry Boehm, *Software Engineering Economics* (1981)

En la ingeniería del software, el **personal** representa el componente más determinante para el éxito de un proyecto. A diferencia de las máquinas o los algoritmos, **las personas son agentes creativos, adaptativos y emocionales**, cuya gestión requiere tanto habilidades técnicas como sociales. De hecho, el informe *Chaos Report* del *Standish Group* (2020) muestra que **el 75% de los proyectos de software fallidos** tienen su raíz en problemas relacionados con la **comunicación, liderazgo o falta de competencias del equipo**, no con la tecnología.

La gestión del personal implica comprender **cómo organizar, motivar, capacitar y liderar** a los miembros del equipo para que trabajen de manera coordinada hacia un mismo objetivo: **entregar software funcional, de calidad y alineado con las necesidades del cliente**.

En palabras de Tom DeMarco y Timothy Lister en *Peopleware* (1987), *"los proyectos de software exitosos no se construyen con las mejores herramientas, sino con las mejores personas."*

Roles y responsabilidades en equipos de desarrollo

Cada integrante de un equipo de software cumple un **rol fundamental**, el cual debe estar **claramente definido** para evitar conflictos, duplicación de esfuerzos o vacíos de responsabilidad.

a) Roles principales en metodologías ágiles (como SCRUM):

<i>Rol</i>	<i>Responsabilidad principal</i>	<i>Habilidades requeridas</i>
<i>Product Owner (PO)</i>	Define el valor del producto, gestiona el <i>backlog</i> y representa al cliente.	Comunicación, visión estratégica, priorización.
<i>Scrum Master (SM)</i>	Facilita el proceso Scrum, elimina impedimentos y promueve la mejora continua.	Liderazgo servicial, resolución de conflictos, conocimiento de Scrum.
<i>Equipo de desarrollo</i>	Diseña, codifica, prueba y entrega incrementos de software funcional.	Programación, diseño, pruebas, trabajo colaborativo.
<i>Tester o QA Engineer</i>	Verifica la calidad y cumplimiento de requisitos.	Pruebas automatizadas, documentación, pensamiento analítico.

Selección y contratación del personal

El proceso de **selección del talento** es tan crucial como el diseño del software. Se debe buscar un equilibrio entre **competencias técnicas (hard skills)** y **competencias blandas (soft skills)**.

a) Competencias técnicas:

- Dominio de lenguajes de programación relevantes (Python, JavaScript, C#, etc.).
- Conocimiento de patrones de diseño, pruebas unitarias, integración continua.
- Capacidad para comprender arquitecturas distribuidas o basadas en microservicios.

b) Competencias blandas:

- Comunicación asertiva.

- Resolución de problemas.
- Adaptabilidad y pensamiento crítico.
- Capacidad de trabajo colaborativo.

Ejemplo:

En Nicaragua, muchas empresas tecnológicas como *INTEGRA*, *Xolotlán Software* o *Comtech* valoran tanto la capacidad técnica como la disposición para **aprender rápidamente** y **trabajar en equipo remoto**, especialmente tras la adopción del teletrabajo post-pandemia.

La **entrevista técnica estructurada**, las **pruebas de código** y la **evaluación de comportamiento (basada en competencias)** son herramientas recomendadas según Pressman & Maxim (2020, *Software Engineering: A Practitioner's Approach*).

Formación y desarrollo profesional

El ritmo de evolución tecnológica obliga a las organizaciones a promover una **formación continua** del personal. La obsolescencia tecnológica es un riesgo real: un desarrollador que no se actualiza cada 2 años puede quedar rezagado frente a los estándares del mercado.

Estrategias efectivas:

- **Capacitación interna:** talleres de actualización tecnológica, revisiones de código conjuntas, *tech talks*.
- **Educación formal:** diplomados, maestrías o certificaciones (p. ej. *Scrum Master Certified*, *AWS Developer Associate*).
- **Aprendizaje colaborativo:** mentorías entre compañeros, comunidades de práctica.

Motivación y liderazgo

Un equipo motivado rinde exponencialmente más que uno desmotivado. La teoría de **Herzberg (1959)** distingue entre:

- **Factores higiénicos:** salario, condiciones laborales, estabilidad.
- **Factores motivadores:** reconocimiento, crecimiento profesional, sentido del logro.

El liderazgo efectivo en ingeniería del software se basa en el **liderazgo servicial (servant leadership)**, donde el líder busca **remover obstáculos y potenciar las capacidades del equipo**, más que imponer autoridad.

Estrategias de motivación:

- Retroalimentación positiva frecuente.
- Reuniones de reconocimiento (*retrospectivas* en Scrum).
- Flexibilidad laboral y autonomía.
- Participación en decisiones técnicas.

Ejemplo local:

En equipos mixtos de universidades y empresas nicaragüenses, la motivación suele fortalecerse cuando los líderes permiten a los estudiantes o desarrolladores **elegir las tecnologías** que se usarán, fomentando un sentido de propiedad y compromiso.

Comunicación efectiva

La **comunicación es el sistema nervioso del equipo de software**. Los malentendidos técnicos o de expectativas pueden causar retrasos, retrabajo y frustración.

Herramientas y prácticas clave:

- **Reuniones diarias (Daily Stand-ups):** breves y centradas en el avance.
- **Documentación compartida:** uso de Notion, Confluence o Google Docs.
- **Canales síncronos y asíncronos:** Slack, Discord, correo electrónico.
- **Revisiones de código conjuntas:** promueven aprendizaje y coherencia técnica.

Ejemplo:

En un proyecto interuniversitario de desarrollo de software educativo, se detectó que el 60% de los errores en el backlog provenían de malentendidos entre los diseñadores y los programadores. Implementar una reunión semanal de revisión de requerimientos redujo los errores en un 45%.

Impacto de la gestión del personal en el éxito del proyecto

La evidencia empírica respalda que **una gestión eficiente del personal reduce los costos y mejora la calidad del software**. Según Boehm y Turner (2004), los equipos altamente cohesionados y con liderazgo efectivo pueden reducir **hasta en un 30% los defectos post-lanzamiento**.

El modelo **CMMI (Capability Maturity Model Integration)** incluye el área de proceso "People Capability Maturity Model (P-CMM)", que formaliza las prácticas para **desarrollar competencias organizacionales basadas en las personas**, lo que demuestra la importancia del recurso humano en la madurez de los procesos.

El personal es **el corazón de la ingeniería del software**. Ninguna metodología, herramienta o infraestructura puede compensar la falta de **talento motivado, capacitado y bien gestionado**.

Gestionar al personal implica un equilibrio entre ciencia y arte: aplicar técnicas de gestión de proyectos, pero también comprender la naturaleza humana, fomentar el aprendizaje continuo y construir una cultura organizacional donde cada individuo se sienta **parte de algo significativo**.

Como diría Frederick Brooks en *The Mythical Man-Month* (1995):

"No se puede acelerar el embarazo de una mujer poniendo a nueve mujeres a trabajar en ello."

— Un recordatorio ingenioso de que el talento humano no es divisible ni sustituible; requiere tiempo, respeto y dirección.

Producto

¿Qué entendemos por “producto” en ingeniería de software?

Producto = el sistema, aplicación o servicio entregable que satisface necesidades concretas de usuarios/organizaciones. Incluye código, documentación, datos, interfaces, procesos de despliegue y acuerdos de servicio (SLA/SLO). No es solo “features”: incorpora requisitos funcionales y **no funcionales** (rendimiento, seguridad, mantenibilidad, usabilidad, compatibilidad, etc.). Esta idea está formalizada en modelos de calidad como **ISO/IEC 25010**, que enumera las características que definen la calidad de un producto software.

Carga conceptual clave: el producto debe pensarse como **un conjunto de hipótesis de valor** que deben validarse con usuarios reales (product-market fit), no sólo como un catálogo de requerimientos. Esta distinción alimenta prácticas de *product discovery*.

Principios de gestión del producto

1. **Propósito y Product Goal:** todo backlog, priorización y release debe enlazarse a un *Product Goal* claro (Scrum lo incorpora explícitamente). El Product Owner es la persona responsable de articularlo y comunicarlo.
2. **Discovery antes que entrega:** validar problemas y soluciones con usuarios antes de invertir grandes esfuerzos en desarrollo (principios de Marty Cagan / SVPG).
3. **Medir resultados (outcomes), no solo outputs:** priorizar por impacto real en usuarios/negocio (ej.: reducción de tiempo de proceso, aumento de retención), no por cantidad de historias entregadas.
4. **Calidad desde el diseño:** incorporar atributos de calidad (ISO 25010) desde la especificación.

Artefactos y prácticas esenciales

Product Backlog — estructura, ítems y criterios

Definición: lista dinámica y priorizada de funcionalidades, mejoras, correcciones y tareas técnicas. El backlog es *vivo*: su prioridad cambia según aprendizaje y contexto.

Formato sugerido para cada ítem (user story):

- **ID:** PB-xxx
- **Título:** Como <rol>, quiero <acción> para <beneficio>
- **Descripción:** 1–3 líneas (contexto)
- **Criterios de aceptación** (mínimo 2–3, preferible en Gherkin)
- **Estimación:** story points (p. ej. Fibonacci)
- **Prioridad:** alta/media/baja o número (WSJF weight)
- **Dependencias:** PB-yyy
- **Definición de Done (DoD):** tests unitarios 80% de cobertura, revisión de código, documentación mínima

Ejemplo (UNP — módulo inscripción):

- Título: *Como estudiante, quiero inscribirme en asignaturas en línea para completar mi matrícula sin filas.*
- Criterios (Gherkin simplificado):
 - Dado un estudiante autenticado, Cuando selecciona asignaturas disponibles y confirma, Entonces su inscripción queda registrada y se muestra comprobante PDF.

Práctica de mantenimiento: refinamiento semanal o cada dos sprints (backlog grooming) con PO + equipo.

Priorización — técnicas prácticas y cómo elegir

- **MoSCoW** (Must/Should/Could/Won't) — simple y pedagógico.
- **WSJF (Weighted Shortest Job First)** — usado en SAFe; prioriza POR impacto económico/usuario dividido por tamaño/duración.
- **Value vs Cost / RICE / Kano / Opportunity Scoring** — según grado de sofisticación.

Plantilla WSJF (simplificada):

- *Valor de negocio* (0–10)
- *Urgencia/Time criticality* (0–10)
- *Riesgo/opportunity enablement* (0–10)
- **Job Size** (esfuerzo estimado)
- **WSJF = (Valor + Urgencia + Riesgo) / Job Size**

Product Discovery — cómo probar hipótesis rápido y barato

Objetivo: comprobar que el problema y la solución propuesta son reales (usable, useful, feasible). Técnicas:

- **Entrevistas con usuarios** (5–10 entrevistas bien dirigidas)
- **Prototipos de baja fidelidad** (paper, Figma rápido)
- **Tests de usabilidad rápidos** (hallazgos accionables en 1–2 días)
- **MVP (Minimum Viable Product)** y experimentos A/B para medir impacto.
- **Métricas de éxito (experimentos):** conversión, tiempo a completar tarea, tasa de error.

SVPG y Marty Cagan recomiendan empoderar equipos cross-funcionales para discovery y delivery continuo.

Ejemplo local: antes de desarrollar un módulo integrado con el “Sistema de Información Académica” de una universidad nicaragüense, hacer un prototipo de pantalla y testear con 10 docentes y 20 estudiantes — medir % de tareas completadas sin error y tiempo medio. Si menos del 70% completan la tarea sin asistencia, rediseñar.

Calidad del producto — modelo y atributos

Usa **ISO/IEC 25010** para definir características de calidad y subcaracterísticas que el producto debe cumplir: **Functional suitability, Reliability, Performance efficiency, Usability, Security, Compatibility, Maintainability, Portability.**

Diseña requisitos no funcionales (NFRs) con métricas concretas: p. ej., *tiempo máximo de respuesta < 2 s en 95% de peticiones, tiempo de recuperación < 30 min.*

Checklist mínima al especificar NFRs:

- Métrica claramente definida (qué, cómo medir).
- Umbral objetivo (SLA/SLO).
- Métodos de verificación (tests automatizados, pruebas de carga, revisión de arquitectura).

Ejemplo técnico (rendimiento): meta: 95th percentile response time < 1.5s bajo 500 usuarios concurrentes; plan: pruebas JMeter/Gatling en pipeline CD.

Testing y aseguramiento de calidad (QA)

QA debe integrarse en todo el ciclo (shift-left):

- **Unit tests** (por componente).
- **Integration tests** (APIs, bases de datos).
- **E2E tests** (Cypress/Selenium).
- **Tests de contrato** si hay microservicios.
- **Pruebas de performance y seguridad** (OWASP Top 10).
- **Prácticas:** TDD/BDD cuando aplique; code review como control de calidad humano.

Automatización: pipeline CI ejecuta tests unitarios + linters; pipeline CD ejecuta pruebas de integración en entornos staging y despliegue controlado a producción.

Referencia práctica: DORA y Four Keys recomiendan métricas que conectan calidad con frecuencia de despliegue y estabilidad.

Entregas, releases y estrategia de versiones

Estrategias de release comunes

- **Release cadenciado (timeboxed):** entregas cada X semanas — predecible para stakeholders.
- **Feature flags / toggles:** desplegar código en producción desactivado; activar cuando listo.
- **Canary releases / blue-green:** minimizar riesgo en despliegues.
- **Dark launches / A/B testing:** probar cambios con subgrupos.

Recomendación para universidades y entornos con usuarios críticos: usar *staging* → *canary* → *producción* y feature flags para activar módulos (p. ej. habilitar módulo matrícula por facultad).

Versionado (Semantic Versioning)

- Formato: **MAJOR.MINOR.PATCH** (semver.org).
 - **MAJOR**: incompatible con versiones previas.
 - **MINOR**: nuevas funcionalidades compatibles.
 - **PATCH**: correcciones de bugs.

Ejemplo: v2.3.1 → major 2, minor 3, patch 1.

Mantenimiento y evolución del producto

Clásicamente se distinguen cuatro tipos de mantenimiento (IEEE / ingeniería de software):

1. **Corrective (correctivo)**: arreglar defectos.
2. **Adaptive (adaptativo)**: ajustar por cambios en entorno (p. ej. nueva versión de SGBD, nueva regulación).
3. **Perfective (perfectivo)**: mejoras solicitadas por usuarios (optimización, UX).
4. **Preventive (preventivo)**: reducir deuda técnica, refactorizaciones.

Política recomendada: reservar un % del sprint (p. ej. 10–20%) para mantenimiento y tech debt; en proyectos con estudiantes, reservar sprint “de mantenimiento” cada 4–6 sprints para consolidar calidad.

Métricas de producto (KPIs) — con fórmulas y ejemplos paso a paso

Selecciona 6–8 KPIs que midan valor, uso, calidad y estabilidad. Aquí tienes los más útiles con fórmulas y ejemplos numéricos (cálculos digit-by-digit):

Métricas de uso / producto

- **DAU (Daily Active Users):** usuarios activos en un día.
- **MAU (Monthly Active Users):** usuarios activos en un mes.
- **DAU/MAU ratio** = $DAU \div MAU \rightarrow$ medida de "engagement".

Ejemplo: DAU = 1,200; MAU = 6,000.

$DAU/MAU = 1,200 \div 6,000$.

Cálculo: $1,200/6,000 = 0.2 \rightarrow 20\%$ (ratio típico para apps con uso periódico).

- **Churn rate (usuarios que dejan de usar)** = $(\text{Usuarios perdidos en periodo} \div \text{Usuarios al inicio del periodo}) \times 100\%$

Ejemplo: inicio mes: 5,000 usuarios; usuarios perdidos: 250.

Cálculo: $250 \div 5,000 = 0.05 \rightarrow 0.05 \times 100\% = 5\%$ churn mensual.

Métricas de calidad / operación (DORA + producto)

- **Deployment Frequency:** despliegues a producción por semana/mes. (mide velocidad).
- **Lead Time for Changes:** tiempo desde commit hasta despliegue. (mide eficiencia).
- **Change Failure Rate:** % de despliegues que requieren hotfix/rollback.
- **MTTR (Mean Time To Recover):** promedio de tiempo para restaurar servicio tras incidente.

Métricas de producto (valor)

- **Conversion rate** (ej.: % de estudiantes que completaron matrícula tras iniciar proceso).
 - Fórmula: $(\text{conversiones} \div \text{visitas_iniciales}) \times 100\%$
- **NPS (Net Promoter Score)** — indicador de satisfacción:

- Encuesta: “¿Qué probabilidad hay de que recomiende el sistema a un colega?” (0–10)
- **Promotores:** 9–10; **Pasivos:** 7–8; **Detractores:** 0–6
- **NPS = %Promotores – %Detractores**

Ejemplo NPS paso a paso: 100 respuestas: 60 promotores, 25 pasivos, 15 detractores.

- $\% \text{Promotores} = 60 \div 100 = 0.60 \rightarrow 60\%$
- $\% \text{Detractores} = 15 \div 100 = 0.15 \rightarrow 15\%$
- $\text{NPS} = 60\% - 15\% = 45$ (NPS = 45). Interpretación: excelente (valores > 30 suelen ser muy buenos).

Seguridad, privacidad y regulaciones (aplicación práctica)

- **Seguridad:** incorporar análisis de amenazas (STRIDE), pruebas de penetración y pipeline SCA (Software Composition Analysis) para dependencias. Integrar OWASP Top 10 como mínimo en la verificación.
- **Privacidad / protección de datos:** definir política de datos, cifrado en reposo y tránsito, control de accesos. En contextos universitarios es crítico proteger datos personales de estudiantes y docentes. Recomendación: revisar la normativa local (si la hay) y aplicar principios de minimización de datos.

Nota: no hago afirmaciones legales específicas sobre leyes nicaragüenses aquí — pero el producto debe contemplar mecanismos técnicos para proteger datos (encriptación, máscaras, consentimientos) y se debe consultar asesoría legal local para cumplimiento normativo.

Tech debt y gobernanza del producto

- **Identificación y contabilización:** registrar deuda técnica con prioridad y costo estimado.
- **Pagos:** plan de amortización (p. ej. 10% del sprint) o sprints dedicados.
- **KPIs:** tiempo medio de entrega vs. tiempo planificado, número de defects atribuibles a deuda técnica.

Ejemplo educativo: en proyectos estudiantiles, exige que cada entrega incluya *una nota sobre deuda técnica* y propuesta de refactor (esto enseña disciplina profesional).

Proceso

Un **proceso** es un conjunto interrelacionado de actividades, roles, entradas, salidas y controles cuyo objetivo es transformar insumos (por ejemplo, requisitos) en productos (software, documentación, servicios). En ingeniería del software, formalizar procesos busca **predecibilidad, calidad, trazabilidad y capacidad de mejora continua**. La norma internacional ISO/IEC/IEEE 12207 establece un marco de procesos para todo el ciclo de vida del software, enfatizando que los procesos son *reutilizables* y *tailorables* según contexto de proyecto u organización.

Carga conceptual clave: pensar en procesos como “contratos operativos” — cada proceso define responsabilidades, entradas, salidas, criterios de aceptación y métricas. Si no hay procesos explícitos, la gobernanza se vuelve ad hoc y los resultados erráticos.

Marco normativo y modelos de referencia

- **ISO/IEC/IEEE 12207 (2017):** marco de procesos para todo el ciclo de vida; organiza procesos en grupos (acuerdo, habilitadores organizacionales, gestión técnica y procesos técnicos). Recomendación seleccionar y *tailorar* procesos a la realidad del proyecto u organización.
- **CMMI / P-CMM:** guías para madurez y mejora continua del proceso organizacional; CMMI define niveles de madurez y prácticas para institucionalizar procesos; el *People CMM* complementa con gestión de talento.

Importante (decisivo): adoptar normas no significa “burocratizar” — significa **documentar, medir y mejorar** de forma adaptada. En la práctica se combina: estándares (12207) + prácticas ágiles + mejora continua (CMMI/PDCA).

Estructura de procesos según ISO/12207

ISO/12207 organiza procesos en 4 grandes grupos; en la práctica conviene mapear esto con tu metodología (Scrum, Kanban o híbrida):

1. **Agreement (Acuerdo):** adquisiciones, suministro — contratos, SLAs.
2. **Organizational project-enabling (Habilitadores organizacionales):** gestión de portafolio, mejora de procesos, infraestructura, capacitación.
3. **Technical management (Gestión técnica):** planificación, seguimiento, configuración, garantía de calidad.
4. **Technical processes (Procesos técnicos):** análisis de requisitos, arquitectura, diseño, implementación, integración, verificación, validación, operación y mantenimiento.

Diseño y modelado de procesos — técnicas prácticas

¿Cómo modelar un proceso?

- **BPMN (Business Process Model and Notation)** para procesos de negocio y flujos que involucran usuarios y sistemas (diagramas legibles para stakeholders).
- **SIPOC** (Suppliers, Inputs, Process, Outputs, Customers) para procesos de alto nivel.
- **Swimlanes** para mostrar responsabilidades por rol (PO, SM, DevOps, QA, Operaciones).

Ejemplo rápido (SIPOC) — proceso *Despliegue de Release*:

- **Suppliers:** Equipo de desarrollo, pipeline CI.
- **Inputs:** Artefacto compilado (build), tests verdes, change request aprobado.
- **Process:** empaquetar → pruebas de integración → despliegue canary → monitorización.
- **Outputs:** release en entorno de producción, nota de versión, rollback planeado.
- **Customers:** usuarios finales, soporte.

Tailoring

ISO/12207 permite *tailoring*: documenta qué procesos se usan, qué actividades se omiten y por qué. Para proyectos pequeños (ej. proyecto estudiantil o PyME local) define un conjunto mínimo (requisitos, diseño, implementación, QA, release, mantenimiento) y registra la justificación del recorte.

Métricas de proceso — cuáles elegir y cómo medir

Selecciona métricas que midan **eficiencia, eficacia y calidad** del proceso. No abuses: 6–8 KPIs primarios son suficientes.

KPIs esenciales de proceso

- **Lead Time (LT):** tiempo desde que una petición entra al proceso hasta su entrega.
- **Cycle Time (CT):** tiempo efectivo de trabajo (excluye espera).
- **Throughput (TP):** número de unidades completadas por periodo (p. ej. historias/sprint).
- **Work In Progress (WIP):** número de ítems en curso.
- **Process Cycle Efficiency (PCE):** proporción de tiempo de valor añadido frente a tiempo total del proceso.

- **First Pass Yield / Quality Rate:** % de ítems que pasan sin retrabajo.
- **Defect Arrival Rate y Defect Resolution Time.**

Fórmulas y ejemplos

Ejemplo A — Ley de Little (para validar coherencia de WIP, Throughput y Lead Time)

Fórmula de Little (aplicable en estado estable):

$$\text{Lead Time} = \text{WIP} / \text{Throughput}$$

Supongamos un equipo con:

- WIP = 12 ítems (historias en curso)
- Throughput promedio = 4 historias completadas por semana

Calculemos Lead Time:

1. WIP = 12
2. Throughput = 4 historias/semana
3. Lead Time = $12 \div 4 = 3$ semanas

Cálculo paso a paso:

- 12 dividido por 4 = 3.

Interpretación: en promedio, una historia tarda 3 semanas desde que entra hasta que se entrega.

Ejemplo B — Process Cycle Efficiency (PCE)

$$\text{PCE} = (\text{Value-Added Time} \div \text{Total Lead Time}) \times 100\%$$

Imagina:

- Tiempo total (Lead Time) = 15 días
- Tiempo de valor añadido (tiempo efectivo de desarrollo y test) = 6 días

Cálculo:

1. Value-Added = 6 días

2. Lead Time = 15 días
3. Fracción = $6 \div 15 = 0.4$
4. PCE = $0.4 \times 100\% = 40\%$

Interpretación: sólo el 40% del tiempo se emplea en actividades productivas; 60% es espera/colas — objetivo de mejora: reducir esperas.

Regla práctica: PCE < 20% indica proceso con mucho desperdicio; PCE entre 20–50% típico en desarrollos; >50% es excelente (difícil sin automatización/flujo continuo).

Ejemplo C — Throughput y mejora esperada

Si aplicas automatización (CI/CD + IaC) y reduces el tiempo de despliegue, el throughput puede subir. Supón throughput actual 4 historias/semana. Tras mejoras se proyecta throughput = 6 historias/semana. Con WIP constante 12:

- Nuevo Lead Time = $12 \div 6 = 2$ semanas (antes 3 semanas).
Ahorro: 1 semana por historia en promedio.

Automatización y cadena de herramientas (toolchain): CI/CD, IaC y GitOps

La automatización es el **motor** para reducir colas, errores humanos y mejorar PCE. Prácticas y recomendaciones:

CI/CD (Integración Continua / Entrega Continua / Despliegue Continuo)

Buenas prácticas:

- **Commit temprano y frecuente.**
- **Build único (build once).**
- **Pipelines como código** (versionados junto al repositorio).

- **Tests automatizados en la pipeline:** unitarios → integración → e2e en staging.
- **Security scans** como pasos automáticos (SCA, SAST).

IaC (Infrastructure as Code)

- Declarar infraestructura en Terraform, Ansible, CloudFormation, etc. Permite reproducibilidad y rollback. Recomendación: **versionar los artefactos IaC y probar en CI.**

GitOps

- Extiende IaC: Git como *single source of truth* para estado de clúster y despliegue; operaciones se ejecutan detectando cambios en repositorio. Útil para Kubernetes y despliegues declarativos.

Arquitectura del pipeline (práctica)

- **Stage 1:** build (compilación, linters)
- **Stage 2:** test unitarios (fast)
- **Stage 3:** integración (DB, servicios simulados)
- **Stage 4:** e2e en staging + pruebas de performance (si aplica)
- **Stage 5:** despliegue canary / blue-green (producción) con monitorización y rollback automático.

Herramientas comunes: GitHub Actions, GitLab CI, Azure Pipelines, Jenkins, ArgoCD (GitOps), Terraform, Ansible. Documenta la política de despliegue y cuándo se hace rollback automático.

Mejora de procesos (SPI) — modelos y prácticas

PDCA / Kaizen

Ciclo clásico: **Plan** → **Do** → **Check** → **Act**. Es la base de la mejora continua y debe integrarse en retros, revisiones de proceso y boards de mejora.

CMMI y appraisals

CMMI ofrece un camino estructurado para institucionalizar prácticas: desde procesos ad-hoc (nivel 1) hasta procesos optimizados (nivel 5). La estrategia práctica: identificar gap analysis → priorizar áreas (gestión de requisitos, verificación, configuración) → plan de mejora incremental.

Métricas de diagnóstico

- **Estabilidad del proceso:** % de cambios de alcance por periodo.
- **Predictibilidad:** comparación entre estimado y real (accuracy de estimaciones).
- **Eficiencia operativa:** PCE, Lead Time.
- **Calidad:** defect density, escape defects (defectos en producción).

Implementa *value-stream mapping* para identificar cuellos de botella y pasos sin valor.

Gobernanza de procesos: roles, artefactos y políticas

Roles clave

- **Process Owner:** responsable del proceso (definición, KPIs, cambios).
- **Process Engineer / Coach:** implementa, entrena y mantiene herramientas.
- **Project Manager / Scrum Master:** garantiza cumplimiento operacional.
- **Security/Compliance Officer:** valida controles regulatorios.

Documenta RACI para cada proceso.

Artefactos mínimos

- **Process Definition Document (PDD):** objetivo, entradas, salidas, roles, flujos, métricas, SLAs.
- **Work Instructions / Checklists:** pasos ejecutables.
- **Registro de cambios del proceso:** versionado y justificación.

Roadmap de implementación de procesos

Fase 0 — Diagnóstico (2–4 semanas)

- Inventario de prácticas actuales, herramientas y personas.
- Entregable: informe de estado y backlog de procesos a mejorar.

Fase 1 — Definición y priorización (4 semanas)

- Seleccionar procesos mínimos (p. ej. control de versiones, CI, release).
- Crear PDDs y RACI.
- Entregable: catálogo de procesos mínimo viable.

Fase 2 — Pilotaje y automatización (8–12 semanas)

- Construir pipelines CI/CD para 1–2 servicios críticos.
- Implementar IaC para entornos dev/staging.

- Entregable: pipelines reproducibles y primera release automatizada.

Fase 3 — Medición y ajuste (8 semanas, iterativo)

- Instrumentar métricas (Lead Time, Throughput, PCE).
- Ejecutar mejoras con PDCA.
- Entregable: dashboard de KPIs y plan de mejora.

Fase 4 — Escalado y gobernanza (continuo)

- Extender pipeline a más equipos, formalizar roles, aspirar a appraisal CMMI si es objetivo estratégico.
- Entregable: manual de gobernanza y plan de formación.

Caso aplicado a Nicaragua — universidad / PyME

Contexto: equipo de desarrollo en la Universidad Politécnica de Nicaragua crea un *Portal de Prácticas Profesionales* para estudiantes y empresas locales.

Diagnóstico inicial

- Sin CI/CD; despliegues manuales en servidor universitario.
- WIP alto (10–15 tickets), throughput $\approx$ 3 historias/semana, Lead Time $\approx$ 4 semanas. (mediciones iniciales).

Objetivos del proceso

- Reducir Lead Time a $\leq$ 2 semanas.
- Implementar CI/CD básico para despliegues repetibles.
- Mejorar PCE del 25% $\rightarrow$ 45% en 3 meses.

Intervenciones concretas

1. **Establecer repositorio Git con ramas protegidas.**
2. **Crear pipeline CI (build + tests unitarios).**
3. **Configurar IaC para entorno staging (terraform básico).**
4. **Implementar proceso de release canary para el portal.**
5. **Instrumentar métricas en dashboard (Grafana/Power BI).**

Resultados esperados (simulados)

- WIP se reduce de 12 → 8; throughput sube de 3 → 5 historias/semana.
- Con WIP=8 y Throughput=5 → Lead Time = $8 \div 5 = 1.6$ semanas (≈ 8 días).
Cálculo: $8 \div 5 = 1.6$.
- PCE mejora por reducción de esperas y despliegue automatizado; de 30% → 48% (medible).

Riesgos locales y mitigación

- **Infraestructura limitada / cortes eléctricos:** usar nube pública con backup y CI en GitHub/GitLab para evitar dependencia local.
- **Baja cultura de automatización:** formación práctica obligatoria (workshops).
- **Restricciones presupuestarias:** usar herramientas open-source (GitLab CE, Jenkins, Terraform).

Proyecto

Proyecto: un esfuerzo temporal, con inicio y fin definidos, destinado a crear un producto, servicio o resultado único. En ingeniería de software un proyecto produce un sistema o conjunto de entregables (software, documentación, formación, infra).

Características clave:

- Temporalidad (inicio-fin)
- Resultado único (no rutinario)
- Restricciones (alcance, tiempo, costo)
- Incertidumbre (tecnológica, de requisitos)
- Necesidad de coordinación multidisciplinaria

Fases del proyecto

Presento un ciclo de vida pragmático (alineado con PMBOK / ISO 21500 adaptado a software):

1. Inicio (Initiation)

- Activos: Project Charter, identificación de stakeholders, análisis de viabilidad básica.
- Entregable clave: *Project Charter* aprobado y sponsor identificado.

2. Planificación (Planning)

- Actividades: definición de alcance, WBS, estimaciones, cronograma, presupuesto, plan de calidad, plan de comunicaciones, plan de riesgos, plan de adquisiciones y RACI.
- Entregable clave: *Project Management Plan* (con todos los subplanes).

3. Ejecución (Execution)

- Desarrollo del producto: sprints/iteraciones, control de calidad, gestión de stakeholders, adquisiciones.

- Entregable clave: incrementos de producto y reportes de avance.

4. Seguimiento y control (Monitoring & Controlling)

- Comparar progreso vs plan (Earned Value, KPIs), gestionar cambios (Change Control Board), controlar riesgos y calidad.
- Entregable clave: reportes periódicos, solicitudes de cambio aprobadas/rechazadas.

5. Cierre (Closure)

- Aceptación formal del producto, entrega de documentación, transferencia operativa, lecciones aprendidas y cierre administrativo/financiero.
- Entregable clave: acta de cierre, lecciones aprendidas.

Documentos esenciales (plantillas resumidas)

3.1 Project Charter

- **Nombre del proyecto**
- **Sponsor o patrocinador**
- **Jefe de proyecto / PM**
- **Fecha de inicio estimada**
- **Fecha de finalización objetivo**
- **Objetivo del proyecto (SMART)**
- **Alcance de alto nivel** (qué incluye / excluye)
- **Beneficios esperados** (KPIs estratégicos)
- **Presupuesto estimado**
- **Principales stakeholders**
- **Riesgos iniciales (Top 3)**
- **Criterios de éxito / aceptación**
- **Aprobación (firma del sponsor)**

Project Management Plan — secciones principales

- Alcance y WBS
- Cronograma y hitos (Gantt)
- Estimaciones y presupuesto (cost baseline)
- Gestión de calidad
- Gestión de riesgos
- Comunicaciones
- Plan de recursos humanos y RACI
- Control de cambios
- Plan de cierre y transferencia

Estimación y planificación detallada

Técnicas de estimación

- **Analogous Estimating** (comparación con proyectos previos)
- **Parametric Estimating** (p. ej. coste por FP, coste por historia)
- **Bottom-up Estimating** (descomponer tareas y estimar individualmente)
- **Planning Poker / Story Points** (para equipos ágiles)
- **Function Points / COSMIC** (medición funcional; útil para contratos)
- **COCOMO II** (modelo paramétrico para estimación de esfuerzo; más formal)

Recomendación práctica para cursos: usar combinación — *story points* para sprints + *bottom-up* para hitos de integración y entregables críticos; presentar una estimación paramétrica simple para justificar presupuesto.

WBS (Work Breakdown Structure)

- Descomposición jerárquica: 1.0 Proyecto → 1.1 Requisitos → 1.2 Diseño → 1.3 Implementación → 1.4 Pruebas → 1.5 Implementación y Soporte.
- Elementos: cada paquete de trabajo debe tener responsable, estimación en horas/días, entregable y criterios de aceptación.

Programación (CPM / PERT)

- **CPM (Critical Path Method)**: identifica ruta crítica (actividades con holgura cero).
- **PERT (Program Evaluation & Review Technique)**: uso de estimaciones optimista (O), más probable (M) y pesimista (P) para calcular duración esperada:
 - $TE = (O + 4M + P) / 6$ (estimación ponderada).
- **Monte Carlo** para análisis probabilístico (en proyectos grandes).

Control del proyecto: Earned Value Management (EVM)

EVM integra alcance, tiempo y coste en métricas cuantitativas. Es imprescindible para control objetivo.

Variables y fórmulas básicas

- **PV (Planned Value)** = valor planeado del trabajo programado (también *BCWS*).
- **EV (Earned Value)** = valor del trabajo realmente realizado (también *BCWP*).
- **AC (Actual Cost)** = costo actual incurrido (también *ACWP*).
- **SV (Schedule Variance)** = $EV - PV$. (positivo = adelantado)
- **CV (Cost Variance)** = $EV - AC$. (positivo = bajo coste)
- **SPI (Schedule Performance Index)** = EV / PV . (>1 = rendimiento mejor que planeado)
- **CPI (Cost Performance Index)** = EV / AC . (>1 = coste por valor eficiente)

- **EAC (Estimate at Completion)** (varias fórmulas; común): $EAC = BAC / CPI$
(BAC = Budget at Completion)

Ejemplo práctico — cálculo paso a paso (digit-by-digit)

Contexto: Proyecto: *Portal de Matrícula CUR (UNP)*.

- **BAC (Budget at Completion)** = C\$180,000 (presupuesto total).
- A la semana 12 (mitad del plan de 24 semanas) la planificación esperaba haber completado 60% del trabajo → **PV** = 60% de BAC = $0.60 \times 180,000$.

Calculamos paso a paso:

1. $PV = 0.60 \times 180,000$
 - $180,000 \times 0.6 = 108,000$.
 - $PV = C\$108,000$.
2. Supongamos que el equipo calcula EV (valor ganado) = 48% del trabajo realmente completado → $EV = 0.48 \times BAC$.
 - $EV = 0.48 \times 180,000$.
 - $180,000 \times 0.48 = (180,000 \times 0.5) - (180,000 \times 0.02)$
 - $180,000 \times 0.5 = 90,000$
 - $180,000 \times 0.02 = 3,600$
 - $90,000 - 3,600 = 86,400$
 - $EV = C\$86,400$.
3. Coste real incurrido (AC) hasta la semana 12: supongamos $AC = C\$95,000$.

Ahora calculamos indicadores:

4. **$SV = EV - PV = 86,400 - 108,000$**
 - $86,400 - 108,000 = -21,600$.
 - **$SV = -C\$21,600$** (negativo → retraso en valor).
5. **$CV = EV - AC = 86,400 - 95,000$**
 - $86,400 - 95,000 = -8,600$.
 - **$CV = -C\$8,600$** (negativo → sobrecoste).

6. **SPI = EV / PV = 86,400 ÷ 108,000**

- $86,400 \div 108,000 =$ reduce zeros: $864 \div 1080 = 0.8$ (porque $864/1080 = 0.8$)
- **SPI = 0.8** → el proyecto avanza al 80% de la velocidad planificada (20% por debajo).

7. **CPI = EV / AC = 86,400 ÷ 95,000**

- $86,400 \div 95,000 \approx 0.909473...$
- Cálculo aproximado paso: $95,000 \times 0.9 = 85,500$; resta 900 → cociente ~ 0.909 .
- **CPI ≈ 0.909** → 90.9% eficiencia de coste (por cada 1 córdoba gastada, se obtiene C\$0.909 de valor ganado).

8. **EAC (estimación al finalizar)** usando $EAC = BAC / CPI$

- $BAC = 180,000$.
- $CPI \approx 0.909$.
- $EAC = 180,000 \div 0.909 \approx 198,000$ (aprox).
- Cálculo: $0.909 \times 198,000 \approx 180,000$. (comprobación)
- **EAC $\approx$ C\$198,000** → sobrecoste proyectado de $\sim$ C\$18,000.

Interpretación: El proyecto está retrasado y gastando más de lo previsto; se proyecta un sobrecoste si no se actúa. Acciones: revisar causas (retrabajo, baja productividad), aplicar medidas correctivas, replanificar alcance o negociar presupuesto.

Gestión de stakeholders y comunicaciones

Identificación y análisis de stakeholders

- **Mapeo:** listar stakeholders, interés (bajo/alto), influencia (baja/alta).
- **Matriz poder/interés:** grupos — gestionar/monitorear/satisfacer/informar.

Plan de comunicaciones (plantilla)

- **Información:** qué reportes (estado, riesgos, decisiones)
- **Formato:** tablero, informe ejecutivo, actas, demos
- **Frecuencia:** diaria/semana/mensual
- **Responsable:** PM / SM / PO
- **Audiencia:** sponsor, comité directivo, usuarios clave

Ejemplo (Portal Matrícula UPOLI):

- Reporte semanal técnico (audiencia: equipo dev, SM)
- Reporte ejecutivo mensual (audiencia: Sponsor, Dirección Académica)
- Demo al final de cada iteración (audiencia: PO, usuarios piloto de un CUR)

Gestión de cambios (Change Management)

- **Control formalizado:** todas las solicitudes de cambio (RFC) pasan por un *Change Control Board (CCB)* o proceso ágil simplificado (PO + PM) para evaluar impacto en tiempo/costo/alcance.
- **Evaluación del cambio:** actualización de PV, EV, AC y recalculo EVM si corresponde.
- **Comunicación y aprobación:** documentar decisiones y actualizar baselines (alcance, cronograma, coste).

Gestión de riesgos

Aunque ya trataste riesgos en espectro administrativo, aquí enfoque operativo:

Registro de riesgos (ejemplo campos)

- ID, Descripción, Causa, Probabilidad (0–1), Impacto (monetario o tiempo), Severidad ($P \times I$), Estrategia (Mitigar/Transferir/Aceptar/Evitar), Acciones de mitigación, Responsable, Estado.

Ejemplo concreto (Portal Matrícula UNP)

- **R-01:** Falta de disponibilidad de docentes para validar requisitos ($P = 0.7$; Impacto = 10 días de retraso $\approx$ C\$6,000).
 - Severidad = $0.7 \times 6,000 = \text{C}\$4,200$.
 - Mitigación: sesiones asincrónicas, incentivos por participación, calendario firmado por Dirección.
- **R-02:** Dependencia con SIG universitario externo ($P = 0.4$; Impacto = 20 días).
 - Mitigación: definir contrato SLA y plan de contingencia (integration mock).

Calidad y aseguramiento (QA) en el proyecto

- **Plan de calidad:** criterios de aceptación, métricas (defect density, escape defects), procesos de revisión de código, pruebas automatizadas y criterios de entrada/salida por fase.
- **Gate checks:** antes de cada release, pasar checklist (DoD) — tests automatizados OK, revisión de seguridad, documentación de usuario mínima, plan de rollback.

Procurement / adquisiciones

- Decidir cuándo externalizar (contratar servicios cloud, consultoría, integraciones).
- Contratos: tipo (FP, T&M), cláusulas de cumplimiento, propiedad intelectual y entregables.
- Evaluación de proveedores (criterios técnicos, económicos y de cumplimiento).

Consejo práctico: en entornos universitarios o PyMEs nicaragüenses, favorecer contratos modulares y pilotos para reducir exposición financiera.

Recursos humanos y gobernanza (RACI aplicado)

Ejemplo RACI (Portal Matrícula)

<i>Actividad</i>	<i>Sponsor</i>	<i>PM</i>	<i>PO</i>	<i>SM</i>	<i>Dev</i>	<i>QA</i>	<i>Dirección CUR</i>
<i>Aprobar Project Charter</i>	A	R	C	I	I	I	C
<i>Priorizar backlog</i>	I	C	R	C	I	I	C
<i>Entrega de sprint</i>	I	C	A	R	R	C	I
<i>Aceptación final</i>	A	R	R	I	I	I	R

(A = Accountable, R = Responsible, C = Consulted, I = Informed)

Cierre del proyecto (Checklist de cierre)

- Aceptación formal del producto por el sponsor/cliente.
- Entregables finales entregados y verificados (código fuente, instaladores, documentación, manuales, SLAs).
- Transferencia de conocimiento: entrenamiento a equipo operativo / soporte.
- Liberación de recursos y finanzas (liquidación de contratos, cierre de proveedores).

- Archivado de documentación (repositorio institucional).
- Retrospectiva de cierre: lecciones aprendidas documentadas y plan de incorporación a procesos futuros.
- Celebración y reconocimiento al equipo (motivación, toque humano — importante).

KPIs recomendados para gestionar proyectos de software

Selecciona 6–8 indicadores primarios (claros y accionables):

1. **EV metrics:** SPI, CPI, SV, CV (para control objetivo).
2. **On-time delivery (%):** % de hitos entregados según plan.
3. **Scope Change Rate:** número de cambios aprobados por periodo.
4. **Defect Escape Rate:** defectos detectados en producción / total defectos.
5. **Team Velocity:** story points por sprint (si se usa Scrum).
6. **Resource Utilization (%):** horas programadas vs horas reales.
7. **Stakeholder Satisfaction:** encuesta periódica (NPS o similar).
8. **Forecasted EAC vs BAC:** proyección de sobrecoste/ahorro.

Bibliografía y Webgrafía

- *Sommerville, I. (2012). Ingeniería del software (9.ª ed.). Pearson / UNAM.*
- *Pressman, R. S., & Maxim, B. R. (2021). Ingeniería del software: Un enfoque práctico (9.ª ed.). McGraw-Hill Interamericana.*
- *Brooks, F. P., Jr. (1995). El mítico hombre-mes: Ensayos sobre ingeniería del software (ed. 20.º aniversario). Addison-Wesley / Editorial técnica (edición en español).*
- *Ries, E. (2012). El método Lean Startup: Cómo crear empresas de éxito utilizando la innovación continua. Deusto.*
- *Cagan, M. (2023). Inspirado: Cómo crear productos tecnológicos que fascinen a los clientes (2.ª ed.). Valor Editions de España.*
- *Project Management Institute. (2017). Guía de los fundamentos para la dirección de proyectos (Guía PMBOK®) (6.ª ed.). Project Management Institute (ed. en español).*
- *Sutherland, J. J. (2016). Scrum: El arte de hacer el doble de trabajo en la mitad de tiempo (ed. en español). Océano / Editorial (edición en español)*

Alcance: Delimitación precisa de los objetivos, entregables y límites de un proyecto de software. Define qué se incluirá y qué no se abordará durante su desarrollo, evitando desviaciones y garantizando una planificación realista.

Calidad del software: Conjunto de atributos que determinan el grado en que un producto de software cumple con los requisitos funcionales y no funcionales, tales como fiabilidad, mantenibilidad, eficiencia y usabilidad.

Ciclo de vida del software: Secuencia estructurada de fases que atraviesa un producto de software desde su concepción hasta su retiro. Incluye actividades como planificación, análisis, diseño, implementación, pruebas, despliegue y mantenimiento.

Gestión de proyectos: Disciplina que integra planificación, organización, dirección y control de los recursos humanos, técnicos y financieros de un proyecto con el propósito de alcanzar los objetivos definidos dentro de las restricciones de tiempo, costo y calidad.

Indicador de desempeño (KPI): Métrica cuantificable utilizada para evaluar el grado de cumplimiento de los objetivos establecidos en un proyecto o proceso. Los KPI permiten medir eficiencia, eficacia y progreso.

Ingeniería del software: Rama de la ingeniería que aplica principios científicos, matemáticos y de gestión al diseño, desarrollo, mantenimiento y evaluación de sistemas de software de alta calidad y fiabilidad.

Línea base: Versión aprobada de un producto o documento que sirve como punto de referencia para futuras actividades de desarrollo o control de cambios.

Planificación: Proceso de definir objetivos, estrategias, recursos, cronogramas y responsabilidades necesarias para la ejecución de un proyecto o fase del ciclo de vida del software.

Proceso de desarrollo: Conjunto estructurado de actividades, métodos, prácticas y herramientas empleadas para crear y mantener un producto de software, asegurando coherencia, trazabilidad y calidad.

Producto de software: Resultado tangible del proceso de desarrollo, compuesto por programas, documentación, datos y configuraciones asociadas, que proporcionan soluciones a necesidades específicas de los usuarios.

Riesgo: Posibilidad de que ocurra un evento o condición incierta que afecte negativamente el logro de los objetivos del proyecto. La gestión de riesgos implica su identificación, análisis, planificación de respuesta y seguimiento.

Scrum: Marco ágil para la gestión de proyectos que enfatiza la colaboración, la entrega incremental de valor y la adaptabilidad mediante ciclos cortos llamados sprints.

Stakeholder (interesado): Persona, grupo u organización que tiene interés o puede influir directa o indirectamente en el éxito o fracaso de un proyecto de software.

Trazabilidad: Capacidad de vincular cada requerimiento con su origen, implementación, pruebas y resultados, garantizando la coherencia entre las fases del ciclo de vida.

Valor entregado: Medida del beneficio real que obtiene el cliente o usuario final tras la entrega de un producto o incremento de software, en relación con sus expectativas y necesidades iniciales.